

DD_SUB-WF-FRAMEWORK-01 - Subscription Workflow Framework

Developer implementation guide: DD_SUB-WF-FRAMEWORK-01-DEV_IMPL_GUIDE-v1.0.md.

MVP simplification baseline: implement the framework once, then phase operation flows. First stabilize activation and revenue-critical operations; add pause/resume/terminate and dunning suspension next; implement upgrade/downgrade/relocation/migration after the core activation/payment/fulfillment path is stable.

1. Purpose

SUB-WF-FRAMEWORK-01 defines the shared orchestration layer used by all subscription lifecycle operations.

A subscription operation is one state-changing or restriction-changing workflow execution against a subscription, for example:

```
activate subscription
pause subscription
resume subscription
upgrade package
downgrade package
relocate HomePass
migrate package + HomePass
apply/remove restriction
suspend for non-payment
terminate subscription
```

This DD gives developers the common implementation contract used by those operations:

- How an operation is recorded.
- How only one unsafe operation runs per subscription.
- How idempotency is enforced.
- How Camunda process instances are correlated with database rows.
- How framework lifecycle events are emitted through Kafka.
- How operator-specific BPMN and configuration are selected.

The operation-specific DDs own the actual business flow for each verb. This framework owns the shared plumbing.

2. Ownership

This DD owns:

- `subscription_operation`, the audit and execution anchor for every subscription operation.
- `kafka_outbox`, the transactional outbox used by workflow components.
- `subscription_operation_config`, the per-operator configuration for operation orchestration.
- Framework REST APIs for operation status, cancel, in-flight lookup, and operation history.
- Framework Kafka events: started, completed, failed, and cancelled.
- Concurrency and idempotency rules used by all operation trigger endpoints.
- BPMN process-key resolution.
- Common timeout and cancellation behavior.

This DD does not own:

- Subscription master state. See SUB-LM-01.
- Worker catalog and worker I/O contracts. See DD_SUB-WF-FRAMEWORK-01-WORKERS.
- BPMN files for each operation. Those belong to operation-specific satellite DDs.
- Activation, pause, resume, upgrade, downgrade, relocation, migration, restrict, suspend-NP, or terminate business rules. Those belong to their satellite DDs.
- Billing charges, proration, invoices, payments, tax, or wallets. See BIL DDs.

- Network provisioning, restriction enforcement, suspension, or disconnection. See FUL DDs.
- Notification preference resolution and delivery retries. See NOT-01.

3. Mental Model

The framework is the operation ledger and coordinator. It does not know every business rule, but it makes every operation observable, cancellable, idempotent, and correlated with Camunda.

```

flowchart LR
    Caller[BO UI / API / Scheduler / Dunning] --> Trigger[Satellite Trigger Endpoint]
    Trigger --> FW[SUB-WF Framework]
    FW --> DB[(subscription_operation)]
    FW --> CFG[(subscription_operation_config)]
    FW --> CAM[Camunda BPMN]
    CAM --> Workers[Workflow Workers]
    Workers --> SUBLM[SUB-LM-01 State APIs]
    Workers --> BIL[BIL Modules]
    Workers --> FUL[FUL Modules]
    Workers --> OUTBOX[(kafka_outbox)]
    OUTBOX --> KAFKA[Kafka]

```

The expected flow is:

1. A caller requests an operation through a satellite trigger endpoint.
2. The trigger endpoint validates auth and request shape.
3. The framework checks idempotency and concurrency.
4. The framework creates one `subscription_operation` row.
5. The framework starts the configured Camunda BPMN process.
6. Workers advance the operation and update `subscription_operation.current_state`.
7. The operation ends in COMPLETED, FAILED, or CANCELLED.
8. Framework lifecycle events are emitted through `kafka_outbox`.

4. Operation Kinds

The framework supports these operation kinds:

Operation kind	Meaning	Owning satellite DD
ACTIVATE	Turn a created subscription into an active service.	SUB-WF-ACTIVATE-01
PAUSE	Voluntarily suspend active service.	SUB-WF-PAUSE-01
RESUME	Resume a suspended service.	SUB-WF-RESUME-01
UPGRADE	Move subscription to a higher package.	SUB-WF-UPGRADE-01
DOWNGRADE	Move subscription to a lower package.	SUB-WF-DOWNGRADE-01
RELOCATION	Move service to another HomePass.	SUB-WF-RELOCATION-01
MIGRATION	Change package and HomePass in one controlled flow.	SUB-WF-MIGRATION-01
RESTRICT	Add or remove active restrictions without changing subscription status.	SUB-WF-RESTRICT-01
SUSPEND_NP	Suspend for non-payment.	SUB-WF-SUSPEND-NP-01
TERMINATE	End the subscription.	SUB-WF-TERMINATE-01

5. Framework State Vocabulary

`subscription_operation.current_state` uses a shared vocabulary so support teams, monitoring, and audit tools can understand operations across all operators.

This vocabulary is not a forced sequence. A minimal BPMN may visit only INITIATED, FULFILLMENT_CALL, and COMPLETED. A richer BPMN may visit validation, billing, payment, fulfillment, user task, and compensation states.

State	Meaning	Terminal?
INITIATED	Operation row was created. BPMN start is being requested.	No
VALIDATING	Rules or eligibility checks are running.	No
PENDING_STATE_FLIP	Workflow is asking SUB-LM to move the subscription to a pending status.	No
BILLING_CALL	Workflow is calling billing for an intent, charge, confirmation, or reversal.	No
AWAITING_PAYMENT	Workflow is waiting for a payment-confirmed message.	No
FULFILLMENT_CALL	Workflow is calling a fulfillment module.	No
AWAITING_FULFILLMENT_RESPONSE	Workflow is waiting for fulfillment completed or failed callback/message.	No
AWAITING_USER_TASK	A Camunda human task is open.	No
COMMITTING_FINAL_STATE	Workflow is committing final subscription status or master fields through SUB-LM.	No
EMITTING_EVENT	Workflow has written an outbox event and is finishing.	No
REVERTING	Workflow is running compensation after a failure or cancellation.	No
COMPLETED	Operation finished successfully.	Yes
FAILED	Operation failed and compensation is complete.	Yes
CANCELLED	Operation was cancelled externally and compensation is complete.	Yes

Operation-specific BPMNs may use sub-states internally, but the database field must stay in this framework vocabulary.

6. Data Model

6.1 subscription_operation

What this table is for:

subscription_operation is the execution record for one subscription operation. It links the API request, subscription, operator, Camunda process instance, current workflow state, final outcome, and audit actor.

Use this table when:

- Starting a subscription operation.
- Checking if a subscription already has an in-flight operation.
- Returning operation status to UI or API callers.
- Correlating Camunda process instances with BSS records.
- Auditing who started an operation and why.
- Listing operation history for a subscription.
- Enforcing idempotency for operation trigger endpoints.

Do not use this table to store operation-specific payloads that only one operation understands. For example, termination cooling-off details belong in the termination DD table, not here.

```
CREATE TABLE subscription_operation (
  operation_id          TEXT PRIMARY KEY,
  operator_code         TEXT NOT NULL,
  subscription_id       TEXT NOT NULL REFERENCES subscription(subscription_id),
  operation_kind        TEXT NOT NULL,

  bpmn_process_key      TEXT NOT NULL,
  bpmn_process_instance_id TEXT NULL,

  initiating_actor_user_id TEXT NOT NULL,
  initiating_actor_role   TEXT NOT NULL,
  initiating_workflow_kind TEXT NULL,
  initiating_workflow_ref  TEXT NULL,
```

```

    idempotency_key          TEXT NOT NULL,
    idempotency_request_hash TEXT NOT NULL,

    prior_subscription_status TEXT NULL,
    current_state            TEXT NOT NULL,
    final_state              TEXT NULL,

    failure_reason_code      TEXT NULL,
    failure_reason_detail    TEXT NULL,

    cancel_reason_code       TEXT NULL,
    cancel_actor_user_id     TEXT NULL,

    started_at               TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at             TIMESTAMPTZ NULL,
    duration_ms              INTEGER NULL,

    created_at               TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at               TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CHECK (operation_kind IN (
        'ACTIVATE',
        'PAUSE',
        'RESUME',
        'UPGRADE',
        'DOWNGRADE',
        'RELOCATION',
        'MIGRATION',
        'RESTRICT',
        'SUSPEND_NP',
        'TERMINATE'
    )),
    CHECK (final_state IS NULL OR final_state IN ('COMPLETED', 'FAILED', 'CANCELLED'))
);

CREATE UNIQUE INDEX uq_subop_idempotency
    ON subscription_operation (operator_code, idempotency_key);

CREATE INDEX idx_subop_subscription
    ON subscription_operation (subscription_id, started_at DESC);

CREATE UNIQUE INDEX uq_subop_in_flight_state_changing
    ON subscription_operation (operator_code, subscription_id)
    WHERE final_state IS NULL AND operation_kind <> 'RESTRICT';

CREATE UNIQUE INDEX uq_subop_in_flight_restrict
    ON subscription_operation (operator_code, subscription_id)
    WHERE final_state IS NULL AND operation_kind = 'RESTRICT';

CREATE INDEX idx_subop_bpmn
    ON subscription_operation (bpmn_process_instance_id)
    WHERE bpmn_process_instance_id IS NOT NULL;

CREATE INDEX idx_subop_kind_state
    ON subscription_operation (operator_code, operation_kind, final_state, started_at DESC);

```

Column descriptions:

Column	Purpose
operation_id	Stable unique ID for the operation. Use a prefixed ULID such as subop_01HZ...
operator_code	Operator that owns this operation, such as WIK, WUG, WTZ, or YASSN.
subscription_id	Subscription being changed. References subscription.subscription_id.
operation_kind	The operation verb. Determines the satellite DD and default BPMN.
bpmn_process_key	Camunda process definition key selected for this operator and operation kind.
bpmn_process_instance_id	Camunda process instance ID after the BPMN starts. Null only before Camunda returns the instance ID.
initiating_actor_user_id	User or service account that requested the operation.
initiating_actor_role	Role/group used for authorization, resolved through FOUNDATION_AUTH.
initiating_workflow_kind	Optional upstream workflow type, such as BIL_04_DUNNING, CUSTOMER_API, BO_UI, CONTRACT_EXPIRY_SCHEDULER, or MANUAL.
initiating_workflow_ref	Optional upstream reference, such as dunning case ID, customer request ID, or scheduler job ID.
idempotency_key	Caller-supplied retry key. Same operator and key must not start two operations.

Column	Purpose
idempotency_request_hash	Hash of the canonical trigger request. Used to detect key reuse with a different request body.
prior_subscription_status	Subscription status before the operation started. Used for compensation and audit.
current_state	Current framework workflow state. Updated as the BPMN progresses.
final_state	Null while in-flight. Set to COMPLETED, FAILED, or CANCELLED when terminal.
failure_reason_code	Machine-readable reason for failure, for example OPERATION_TIMEOUT or FULFILLMENT_FAILED.
failure_reason_detail	Short human-readable failure detail. Do not store secrets, stack traces, or full external payloads.
cancel_reason_code	Machine-readable cancellation reason when final state is CANCELLED.
cancel_actor_user_id	User or service account that requested cancellation.
started_at	Time the framework accepted the operation.
completed_at	Time the operation became terminal.
duration_ms	completed_at - started_at in milliseconds. Useful for SLA reporting.
created_at	Row creation timestamp.
updated_at	Last row update timestamp.

Sample data:

operation_id	operator_code	subscription_id	operation_kind	bpmn_process_key	current_state	final_state	initiating_actor_role
subop_01HZ_KAMAU_ACT	WIK	sub_01HZ_KAMAU_FIBER	ACTIVATE	sub-activate-wik	COMPLETED	COMPLETED	CALL_CENTER
subop_01HZ_NJERI_SUSPNP	WIK	sub_01HZ_NJERI_TRIPLEPLAY	SUSPEND_NP	sub-suspend-np-wik	AWAITING_FULFILLMENT_RESPONSE	NULL	BILLING_INTERNAL
subop_01HZ_DAKAR_TERM	YASSN	sub_01HZ_DAKAR_001	TERMINATE	sub-terminate-yassn	COMPLETED	COMPLETED	CALL_CENTER

Sample meaning:

- subop_01HZ_KAMAU_ACT is a completed WIK activation started by a call-center actor.
- subop_01HZ_NJERI_SUSPNP is still running because final_state is null. The BPMN is waiting for fulfillment response.
- subop_01HZ_DAKAR_TERM shows that YASSN can use the same framework with a different BPMN process key.

6.2 kafka_outbox

What this table is for:

kafka_outbox stores Kafka messages inside the same database transaction as the business change. A separate dispatcher publishes those messages to Kafka.

Use this table when:

- An operation changes database state and must emit an event.
- A worker must guarantee that a database mutation and Kafka event are not separated.
- A Kafka broker outage should not roll back the business transaction.
- Failed event delivery needs retry and alerting.

Do not publish directly to Kafka inside the same code path that writes domain state. Write the outbox row first, commit, and let the dispatcher publish asynchronously.

```
CREATE TABLE kafka_outbox (
  id          TEXT PRIMARY KEY,
  topic       TEXT NOT NULL,
  message_key TEXT NOT NULL,
  event_type  TEXT NOT NULL,
  aggregate_id TEXT NOT NULL,
  payload     JSONB NOT NULL,
```

```

        status          TEXT NOT NULL DEFAULT 'PENDING',
        attempts        INTEGER NOT NULL DEFAULT 0,
        next_attempt_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        last_attempt_at TIMESTAMPTZ NULL,
        last_error       TEXT NULL,

        created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        published_at     TIMESTAMPTZ NULL,

        CHECK (status IN ('PENDING', 'PUBLISHED', 'FAILED'))
    );

    CREATE INDEX idx_outbox_pending
    ON kafka_outbox (status, next_attempt_at, created_at)
    WHERE status = 'PENDING';

    CREATE INDEX idx_outbox_failed
    ON kafka_outbox (created_at DESC)
    WHERE status = 'FAILED';

```

Column descriptions:

Column	Purpose
id	Unique outbox row ID. Use a prefixed ULID such as kox_01HZ...
topic	Kafka topic to publish to.
message_key	Kafka message key. For subscription events, use subscription_id so ordering is stable per subscription.
event_type	Event name, such as SubscriptionOperationStarted.
aggregate_id	Main aggregate ID for the event. For framework operation events, use operation_id.
payload	Full JSON event payload to publish.
status	Dispatch status: PENDING, PUBLISHED, or FAILED.
attempts	Number of publish attempts already made.
next_attempt_at	Earliest time the dispatcher should retry this row.
last_attempt_at	Last publish attempt timestamp.
last_error	Short publish error summary. Do not store credentials or huge stack traces.
created_at	Time the event was created.
published_at	Time the event was successfully published.

Sample data:

id	topic	message_key	event_type	aggregate_id	status	attempts
kox_01HZ_001	sophix.subscription.operation.started	sub_01HZ_NJERI_TRIPLEPLAY	SubscriptionOperationStarted	subop_01HZ_NJERI_SUSPNP	PUBLISHED	1
kox_01HZ_002	sophix.subscription.operation.failed	sub_01HZ_KAMAU_FIBER	SubscriptionOperationFailed	subop_01HZ_KAMAU_TERM	PENDING	0
kox_01HZ_003	sophix.subscription.operation.completed	sub_01HZ_DAKAR_001	SubscriptionOperationCompleted	subop_01HZ_DAKAR_TERM	FAILED	10

Sample meaning:

- kox_01HZ_001 has already been published.
- kox_01HZ_002 is waiting for the dispatcher.
- kox_01HZ_003 failed after repeated attempts and must generate an operational alert.

6.3 subscription_operation_config

What this table is for:

subscription_operation_config stores the default orchestration settings for each (operator_code, operation_kind) pair.

Use this table when:

- Resolving which BPMN process key to start.
- Applying operation timeout values.
- Applying billing or fulfillment timeout values.
- Enabling or disabling operator-specific feature flags.
- Changing operator behavior without redeploying BSS code.

Do not use this table for detailed business rules. Put rule logic in Drools packages owned by the relevant satellite DD.

```
CREATE TABLE subscription_operation_config (
  operator_code          TEXT NOT NULL,
  operation_kind         TEXT NOT NULL,
  default_bpmn_process_key TEXT NOT NULL,
  operation_timeout_seconds INTEGER NOT NULL DEFAULT 90,
  billing_call_timeout_seconds INTEGER NOT NULL DEFAULT 30,
  fulfillment_call_timeout_seconds INTEGER NOT NULL DEFAULT 60,
  feature_flags          JSONB NOT NULL DEFAULT '{}',
  enabled               BOOLEAN NOT NULL DEFAULT TRUE,
  updated_at            TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_by            TEXT NOT NULL,

  PRIMARY KEY (operator_code, operation_kind),
  CHECK (operation_kind IN (
    'ACTIVATE',
    'PAUSE',
    'RESUME',
    'UPGRADE',
    'DOWNGRADE',
    'RELOCATION',
    'MIGRATION',
    'RESTRICT',
    'SUSPEND_NP',
    'TERMINATE'
  ))
);
```

Column descriptions:

Column	Purpose
operator_code	Operator this config applies to.
operation_kind	Operation kind this config applies to.
default_bpmn_process_key	Camunda process definition key to start by default.
operation_timeout_seconds	Maximum operation duration before the BPMN timeout path must run.
billing_call_timeout_seconds	Default timeout for billing worker calls for this operation.
fulfillment_call_timeout_seconds	Default timeout for fulfillment worker calls for this operation.
feature_flags	Operator/kind feature switches consumed by BPMN or Drools.
enabled	If false, trigger endpoints must reject new operations for this operator/kind.
updated_at	Last config update timestamp.
updated_by	User or service account that last changed the config.

Sample data:

operator_code	operation_kind	default_bpmn_process_key	operation_timeout_seconds	enabled	feature_flags
WIK	ACTIVATE	sub-activate-wik	120	true	{"customer_self_service_enabled": false}
WIK	TERMINATE	sub-terminate-wik	120	true	{"cooling_off_supervisor_override_required": true}
WIK	SUSPEND_NP	sub-suspend-np-wik	90	true	{}
YASSN	TERMINATE	sub-terminate-yassn	90	true	{"customer_self_service_enabled": false}

Sample meaning:

- WIK activation uses the WIK activation BPMN and a 120-second operation timeout.

- WIK termination requires the termination BPMN and has a cooling-off supervisor flag available to rules.
- YASSN termination uses a different BPMN key without changing framework code.

7. Business Rules

Rule	Description
R-SUB-WF-FW-1	At most one in-flight state-affecting operation may run for one (operator_code, subscription_id).
R-SUB-WF-FW-2	One RESTRICT may run concurrently with one non-RESTRICT operation only if the satellite DD confirms it does not change status_code. Two RESTRICT operations must not run concurrently on the same subscription.
R-SUB-WF-FW-3	TERMINATE may interrupt an in-flight non-TERMINATE operation by sending a Camunda cancel message to the old operation, waiting for cancellation handling, then starting termination.
R-SUB-WF-FW-4	Idempotency is enforced on (operator_code, idempotency_key) for every trigger endpoint.
R-SUB-WF-FW-5	Reusing the same idempotency key with a different canonical request hash returns 409 IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST.
R-SUB-WF-FW-6	Trigger endpoints are asynchronous by default and return 202 Accepted with operation_id.
R-SUB-WF-FW-7	The BPMN process key is resolved from subscription_operation_config. Non-production callers may override it only when explicitly allowed by environment configuration.
R-SUB-WF-FW-8	The framework must set bpmn_process_instance_id as soon as Camunda returns it.
R-SUB-WF-FW-9	Operation timeout is enforced by BPMN boundary timer using operation_timeout_seconds.
R-SUB-WF-FW-10	On timeout, the operation enters REVERTING, then ends in FAILED with failure_reason_code = OPERATION_TIMEOUT.
R-SUB-WF-FW-11	Workers must be operator-agnostic. Operator-specific differences belong in BPMN, Drools, configuration, and auth groups.
R-SUB-WF-FW-12	Domain mutations and Kafka events must use the outbox pattern.
R-SUB-WF-FW-13	Notification failure must not fail a subscription operation. Notification retries belong to NOT-01.
R-SUB-WF-FW-14	Every satellite trigger endpoint must enforce auth before it calls the framework start logic. Drools may re-check authorization for defense in depth.

8. REST APIs

The framework owns cross-operation APIs. Operation trigger endpoints such as POST /api/subscriptions/{subscription_id}/terminate are owned by satellite DDs, but they must call the framework start contract.

8.1 Start Operation Contract Used By Satellites

This is an internal service contract, not necessarily a public REST endpoint.

Input:

```
{
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_NJERI_TRIPLEPLAY",
  "operationKind": "SUSPEND_NP",
  "idempotencyKey": "dunning-case-8743-suspend",
  "initiatingActorUserId": "svc-bil-04",
  "initiatingActorRole": "BILLING_INTERNAL",
  "initiatingWorkflowKind": "BIL_04_DUNNING",
  "initiatingWorkflowRef": "dun_8743",
  "requestPayload": {
    "reasonCode": "NON_PAYMENT",
    "source": "DUNNING"
  }
}
```

Success:

```
{
```



```

    "operationId": "subop_01HZ_NJERI_SUSPNP",
    "operationKind": "SUSPEND_NP",
    "currentState": "INITIATED",
    "finalState": null,
    "bpmnProcessKey": "sub-suspend-np-wik",
    "bpmnProcessInstanceId": "pi_01HZ_NJERI_SUSPNP"
  }
}

```

8.2 Cancel Operation

POST /api/subscription-operations/{operation_id}/cancel HTTP/1.1
 Authorization: Bearer <bo-admin-jwt>
 Content-Type: application/json

```

{
  "cancelReason": "CUSTOMER_CHANGED_MIND",
  "cancelActorUserId": "bo-admin-007"
}

```

Success:

```

{
  "operationId": "subop_01HZ_NJERI_SUSPNP",
  "previousState": "FULFILLMENT_CALL",
  "newState": "CANCELLED",
  "cancelledAt": "2026-05-22T18:30:00+03:00",
  "subscriptionRevertedTo": "ACTIVE"
}

```

Already terminal:

```

{
  "code": "OPERATION_ALREADY_TERMINAL",
  "finalState": "COMPLETED",
  "completedAt": "2026-05-22T18:00:00+03:00"
}

```

8.3 Lookup Operation Status

GET /api/subscription-operations/{operation_id} HTTP/1.1
 Authorization: Bearer <jwt>

Response:

```

{
  "operationId": "subop_01HZ_KAMAU_TERM",
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operationKind": "TERMINATE",
  "bpmnProcessKey": "sub-terminate-wik",
  "bpmnProcessInstanceId": "pi_01HZ_KAMAU_TERM",
  "currentState": "COMPLETED",
  "finalState": "COMPLETED",
  "startedAt": "2026-07-20T14:30:00+03:00",
  "completedAt": "2026-07-20T14:30:18+03:00",
  "durationMs": 18432
}

```

8.4 In-Flight Operation Lookup

GET /api/subscriptions/{subscription_id}/in-flight-operation HTTP/1.1
 Authorization: Bearer <jwt>

When one exists:

```

{
  "operationId": "subop_01HZ_NJERI_SUSPNP",
  "operationKind": "SUSPEND_NP",
  "currentState": "AWAITING_FULFILLMENT_RESPONSE",
  "startedAt": "2026-05-22T18:28:00+03:00"
}

```

When none exists:

204 No Content

8.5 Operation History

GET /api/subscriptions/{subscription_id}/operations?since=2026-01-01 HTTP/1.1
 Authorization: Bearer <jwt>

Response:

```

{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "operations": [
    {
      "operationId": "subop_01HZ_KAMAU_ACT",
      "operationKind": "ACTIVATE",
      "finalState": "COMPLETED",
      "completedAt": "2026-01-15T08:30:18+03:00"
    },
    {
      "operationId": "subop_01HZ_KAMAU_TERM",
      "operationKind": "TERMINATE",
      "finalState": "COMPLETED",
      "completedAt": "2026-07-20T14:30:18+03:00"
    }
  ]
}

```

9. Kafka Events

Framework events are operation lifecycle markers. They do not replace operation-specific domain events such as `SubscriptionActivated` or `SubscriptionTerminated`.

Topic prefix:

`sophix.subscription.operation.*`

Event	Topic	When emitted	Message key
<code>SubscriptionOperationStarted</code>	<code>sophix.subscription.operation.started</code>	After operation row is created and BPMN start is requested.	<code>subscription_id</code>
<code>SubscriptionOperationCompleted</code>	<code>sophix.subscription.operation.completed</code>	When <code>final_state = COMPLETED</code> .	<code>subscription_id</code>
<code>SubscriptionOperationFailed</code>	<code>sophix.subscription.operation.failed</code>	When <code>final_state = FAILED</code> .	<code>subscription_id</code>
<code>SubscriptionOperationCancelled</code>	<code>sophix.subscription.operation.cancelled</code>	When <code>final_state = CANCELLED</code> .	<code>subscription_id</code>

Sample `SubscriptionOperationCompleted`:

```

{
  "eventType": "SubscriptionOperationCompleted",
  "aggregateId": "subop_01HZ_KAMAU_TERM",
  "timestamp": "2026-07-20T14:30:18+03:00",
  "payload": {
    "operationId": "subop_01HZ_KAMAU_TERM",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "operationKind": "TERMINATE",
    "bpmnProcessKey": "sub-terminate-wik",
    "finalState": "COMPLETED",
    "startedAt": "2026-07-20T14:30:00+03:00",
    "completedAt": "2026-07-20T14:30:18+03:00",
    "durationMs": 18432
  }
}

```

10. Auth

Framework APIs must use `FOUNDATION_AUTH`.

Minimum rules:

- BO/admin users may look up operation status and history for their operator scope.
- Only allowed admin roles may cancel operations.
- Service accounts may start operations only for their permitted workflow kind.
- Satellite trigger endpoints must validate role before calling the framework start contract.
- The framework must not trust `initiating_actor_role` supplied by a caller without resolving it from the authenticated token or service account.

Candidate group and user-task authorization remain operator-specific and are configured through `FOUNDATION_AUTH` and BPMN user-task definitions.

11. Error Handling

Error code	HTTP status	When
OPERATION_NOT_FOUND	404	Operation ID does not exist or caller cannot access it.
OPERATION_IN_PROGRESS	409	Another non-compatible operation is already in-flight for the subscription.
OPERATION_ALREADY_TERMINAL	409	Cancel was requested for an operation already completed, failed, or cancelled.
IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST	409	Same (operator_code, idempotency_key) used with different request hash.
OPERATION_KIND_DISABLED	409	Config row exists but enabled = false.
OPERATION_CONFIG_NOT_FOUND	500 or deployment error	Missing config for (operator_code, operation_kind). Should be caught during deployment validation.
BPMN_START_FAILED	502	Camunda start request failed after operation row was created.
OPERATION_TIMEOUT	Terminal failure	BPMN boundary timer elapsed.
UNAUTHORIZED_OPERATION_ROLE	403	Actor role is not allowed to trigger or cancel this operation.

12. Implementation Notes

12.1 Trigger Endpoint Sequence

Every satellite trigger endpoint should follow the same sequence.

1. Authenticate the caller.
2. Resolve operator_code, actor user ID, and actor role from auth context.
3. Validate request fields.
4. Validate the actor role is allowed for this operation kind.
5. Canonicalize the request body.
6. Hash the canonical request body.
7. Start a database transaction.
8. Lock the subscription row in SUB-LM using SELECT ... FOR UPDATE or the local repository equivalent.
9. Load subscription_operation_config for (operator_code, operation_kind).
10. Reject if config is missing or disabled.
11. Check idempotency by (operator_code, idempotency_key).
12. Check for existing in-flight operation.
13. If allowed, insert subscription_operation in INITIATED.
14. Insert SubscriptionOperationStarted into kafka_outbox.
15. Commit the database transaction.
16. Start the Camunda BPMN process.
17. Save bpmn_process_instance_id.
18. Return 202 Accepted.

Do not call Camunda while holding a row lock on subscription. Keep the lock only long enough to reserve the operation.

12.2 Idempotency

Idempotency prevents network retries from starting duplicate operations.

Use subscription_operation itself as the idempotency record:

```
CREATE UNIQUE INDEX uq_subop_idempotency
```

```
ON subscription_operation (operator_code, idempotency_key);
```

Implementation behavior:

1. Compute `idempotency_request_hash` from canonical JSON.
2. Try to insert the operation row.
3. If insert succeeds, this request owns the operation.
4. If insert fails because `uq_subop_idempotency` already exists, load the existing row.
5. If the existing hash differs, return `409 IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST`.
6. If the existing hash matches, return the existing `operation_id`, `current_state`, and `final_state`.

Important: operation insert can also fail because an in-flight-operation unique index was violated. That is not an idempotency result. In that case, return `409 OPERATION_IN_PROGRESS` with the current operation details.

Example pseudocode:

```
OperationStartResult start(StartOperationCommand cmd) {
    String hash = hash(canonicalJson(cmd.requestPayload()));

    try {
        Operation op = insertOperation(cmd, hash);
        return startNewOperation(op);
    } catch (UniqueConstraintViolation e) {
        if (e.constraintName().equals("uq_subop_idempotency")) {
            throw operationInProgress(cmd.operatorCode(), cmd.subscriptionId());
        }

        Operation existing = operationRepo.findByIdempotencyKey(
            cmd.operatorCode(),
            cmd.idempotencyKey()
        );

        if (!existing.idempotencyRequestHash().equals(hash)) {
            throw conflict("IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST");
        }

        return resultFromExistingOperation(existing);
    }
}
```

Do not make idempotency optional. BO UI, dunning jobs, and schedulers will retry after timeouts.

12.3 In-Flight Operation Check

Use the partial unique index as the database guarantee:

```
CREATE UNIQUE INDEX uq_subop_in_flight_state_changing
ON subscription_operation (operator_code, subscription_id)
WHERE final_state IS NULL AND operation_kind <> 'RESTRICT';

CREATE UNIQUE INDEX uq_subop_in_flight_restrict
ON subscription_operation (operator_code, subscription_id)
WHERE final_state IS NULL AND operation_kind = 'RESTRICT';
```

Application code should still check first so it can return a good error:

```
{
  "code": "OPERATION_IN_PROGRESS",
  "currentOperationId": "subop_01HZ_NJERI_SUSPNP",
  "currentOperationKind": "SUSPEND_NP",
  "currentState": "AWAITING_FULFILLMENT_RESPONSE"
}
```

The indexes protect against race conditions where two requests pass the application check at the same time. They allow at most one state-changing operation and at most one RESTRICT operation per subscription. This gives the satellite DD room to allow a restriction change during another operation, while still preventing two restriction updates from overwriting each other.

12.4 TERMINATE Interruption

TERMINATE is special because the business may require termination even if another operation is running.

Recommended sequence:

1. Lock the subscription.
2. Find the current in-flight operation.

3. If no operation exists, start termination normally.
4. If current operation is TERMINATE, return OPERATION_IN_PROGRESS.
5. If current operation is not TERMINATE, send Camunda message external-cancel-requested to the existing process.
6. Existing BPMN moves to REVERTING.
7. Existing BPMN marks itself CANCELLED.
8. Start the new termination operation.

Do not mark the old operation CANCELLED directly unless Camunda cannot be reached and the operational runbook explicitly allows manual repair. The BPMN owns compensation.

12.5 Starting Camunda Safely

The operation row is created before Camunda starts. That means a small failure window exists:

```
DB commit succeeds, but Camunda start fails
```

Handle this explicitly:

- If Camunda start succeeds, update bpmn_process_instance_id.
- If Camunda start fails, mark the operation FAILED with failure_reason_code = BPMN_START_FAILED.
- Insert SubscriptionOperationFailed into kafka_outbox.
- Return 502 BPMN_START_FAILED to the caller.

Do not leave operations stuck forever in INITIATED without a process instance. Add a scheduled repair job that finds old INITIATED rows where bpmn_process_instance_id IS NULL.

12.6 Updating Operation State

Workers update subscription_operation.current_state as they pass major BPMN milestones.

Example:

```
UPDATE subscription_operation
SET current_state = 'FULFILLMENT_CALL',
    updated_at = NOW()
WHERE operation_id = :operationId
    AND final_state IS NULL;
```

Rules:

- Never update current_state after final_state is set.
- Only use framework vocabulary values.
- Keep operation-specific detail in satellite tables or Camunda variables.

12.7 Completing an Operation

Completion must be atomic with outbox insert.

```
UPDATE subscription_operation
SET current_state = 'COMPLETED',
    final_state = 'COMPLETED',
    completed_at = NOW(),
    duration_ms = EXTRACT(EPOCH FROM (NOW() - started_at)) * 1000,
    updated_at = NOW()
WHERE operation_id = :operationId
    AND final_state IS NULL;
```

In the same transaction, insert SubscriptionOperationCompleted into kafka_outbox.

If the update affects zero rows, the operation is already terminal. The worker should not emit a duplicate completion event.

12.8 Failing an Operation

Failure must also be atomic with outbox insert.

Required fields:

- current_state = 'FAILED'

- final_state = 'FAILED'
- failure_reason_code
- failure_reason_detail
- completed_at
- duration_ms

Keep failure_reason_detail short. It is for support and audit, not raw exception storage.

12.9 Cancelling an Operation

Cancel API should not directly modify subscription state. It should tell Camunda to cancel and let BPMN compensation run.

Cancel API sequence:

1. Authenticate cancel actor.
2. Load operation.
3. Reject if operation is terminal.
4. Send Camunda message external-cancel-requested with cancel reason and actor.
5. Return accepted/success response.
6. BPMN compensates and marks operation CANCELLED.

12.10 Outbox Dispatcher

The dispatcher reads PENDING rows ordered by created_at.

Suggested behavior:

1. Fetch a small batch, for example 100 rows.
2. Lock rows with FOR UPDATE SKIP LOCKED.
3. Publish each row to Kafka.
4. If publish succeeds, set status = 'PUBLISHED' and published_at = NOW().
5. If publish fails, increment attempts, store last_error, and set next_attempt_at using backoff.
6. After 10 failures, set status = 'FAILED' and raise an ICN-01 alert.

Example query:

```
SELECT *
FROM kafka_outbox
WHERE status = 'PENDING'
      AND next_attempt_at <= NOW()
ORDER BY created_at
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

12.11 Transaction Boundaries

Good transaction:

```
begin
  lock subscription
  insert subscription_operation
  insert kafka_outbox started event
commit

start Camunda outside the lock
update operation with process instance id
```

Bad transaction:

```
begin
  lock subscription
  call Camunda
  call billing
  call fulfillment
  insert operation
commit
```

External calls while holding database locks cause slow requests, deadlocks, and inconsistent retry behavior.

13. Infrastructure Recommendations

13.1 On-Prem Medium Deployment

Deployment unit clarification:

- SUB-LM-01 from the previous DD is the Subscription Lifecycle Management service/module. It owns the subscription master table and low-level subscription APIs.
- SUB-WF-FRAMEWORK-01 in this DD is the Subscription Workflow Orchestration service/module. It owns operation rows, operation status APIs, cancel APIs, idempotency/concurrency checks, Camunda process start, and outbox dispatch.
- In a microservice deployment, these can be two separate services: subscription-management-service for SUB-LM and subscription-workflow-service for SUB-WF.
- In a budget-conscious deployment, they may run inside the same application artifact or the same Kubernetes deployment, but the logical ownership must remain separate in code packages, database migrations, and API routing.

So when this section says "SUB-WF service", it means the operation orchestration module from this DD, not the SUB-LM management module from the previous DD.

For a budget-conscious medium deployment, run the SUB-WF service as stateless application pods and rely on the foundation services for persistence and orchestration.

Recommended starting point:

Component	Count	Size per node/pod	Notes
SUB-LM API pods	2	2 vCPU, 4 GB RAM	Previous DD. Handles subscription master create/read/internal mutation APIs. Can be packaged with SUB-WF for budget, but remains a separate logical module.
SUB-WF operation API pods	2	2 vCPU, 4 GB RAM	This DD. Handles operation start contract, status, history, cancel, idempotency, and concurrency.
SUB-WF outbox dispatcher pods	2	1 vCPU, 2 GB RAM	This DD. Publishes kafka_outbox rows. Use FOR UPDATE SKIP LOCKED; safe to run more than one.
Camunda workers	See WORKERS DD	Size by worker type	Worker sizing belongs to the companion worker DD.
PostgreSQL	Foundation DB sizing	6 nodes retained	DB is critical and already dimensioned in FOUNDATION_DB.
Kafka	Foundation Kafka sizing	4 nodes	Used for event emission.
Camunda	Foundation Camunda sizing	3 app nodes recommended	Workflow engine dependency.

Budget option:

- One deployable app:
- SUB-LM routes and code package
 - SUB-WF operation routes and code package
 - SUB-WF outbox dispatcher process

- Still keep:
- separate database migrations by DD
 - separate service classes/modules
 - separate API route groups
 - separate metrics labels

This reduces infrastructure cost but keeps the design understandable and allows later separation when traffic grows.

OS recommendation:

- Ubuntu Server 24.04 LTS or Red Hat Enterprise Linux 9.
- Use the same OS baseline as the foundation bundle.
- Keep NTP/chrony enabled. Operation timing and timeout audit depend on accurate clocks.

Recommended partitions for application hosts:

Mount	Size	Purpose
/	40 GB	OS and base packages.
/var/log	50 GB	Application logs and audit troubleshooting.
/opt/sophix	20 GB	App binaries/config, if not fully containerized.
/var/lib/containerd or /var/lib/docker	80 GB	Container images and runtime layers.

Keep application logs out of the root partition so noisy workflow failures do not fill /.

13.2 AWS Deployment

Recommended AWS baseline:

- Run SUB-LM APIs, SUB-WF operation APIs, and SUB-WF outbox dispatchers on EKS or ECS Fargate.
- For limited budget, deploy SUB-LM and SUB-WF in one ECS service or one EKS deployment, then split them later if load or ownership requires it.
- Use Amazon RDS PostgreSQL or the existing managed DB pattern from FOUNDATION_DB.
- Use Amazon MSK if Kafka is self-managed by the platform, or keep the foundation Kafka cluster if already standardized.
- Run Camunda according to FOUNDATION_CAMUNDA AWS guidance.
- Use CloudWatch metrics and alarms for operation failures, stuck operations, outbox failures, and Camunda start failures.

Suggested AWS alarms:

Metric	Alarm condition
In-flight operations older than timeout	Any row older than configured timeout plus grace period.
INITIATED without process instance	Any row older than 2 minutes.
Outbox failed rows	<code>count(status = FAILED) > 0</code> .
Outbox pending age	Oldest pending row older than 5 minutes.
Operation failure rate	Failure percentage above agreed SLA threshold.

14. Concrete Camunda REST Mechanics

SUB-WF uses FOUNDATION_CAMUNDA for the runtime standard, but subscription workflow developers should not have to infer the basic REST calls. Use these mechanics in every satellite unless the satellite explicitly overrides a variable name.

14.1 Start A Process Instance

After the operation row is inserted and committed in INITIATED, the framework starts the configured BPMN process:

```
POST /engine-rest/process-definition/key/subscription-activate-wik/start
Content-Type: application/json

{
  "businessKey": "subop_01HZ_ACTIVATE_001",
  "variables": {
    "operationId": { "value": "subop_01HZ_ACTIVATE_001", "type": "String" },
    "subscriptionId": { "value": "sub_01HZ", "type": "String" },
    "operatorCode": { "value": "WIK", "type": "String" },
    "operationKind": { "value": "ACTIVATE", "type": "String" },
    "accountId": { "value": "acc_01HZ_HOME", "type": "String" },
    "customerId": { "value": "cust_01HZ", "type": "String" }
  }
}
```

On success, store returned id as `subscription_operation.bpmn_process_instance_id`. If Camunda returns an error, mark the operation FAILED with `failure_code = CAMUNDA_START_FAILED` and keep the operation row for audit/repair.

14.2 Worker Fetch And Lock

Each consolidated subscription worker app polls the topics it owns:

```
POST /engine-rest/external-task/fetchAndLock
Content-Type: application/json

{
```



```

"workerId": "sub-worker-01",
"maxTasks": 5,
"usePriority": true,
"topics": [
  {
    "topicName": "sub-lm-put-state",
    "lockDuration": 60000,
    "variables": ["operationId", "subscriptionId", "targetStatus", "reasonCode"]
  },
  {
    "topicName": "kafka-emit",
    "lockDuration": 30000,
    "variables": ["operationId", "eventKind", "eventPayload"]
  }
]
}

```

The worker must validate all required variables before calling downstream APIs. Missing variables are BPMN/modeling defects and should fail the task with a clear error message.

14.3 Complete A Service Task

After the worker side effect succeeds:

```

POST /engine-rest/external-task/{externalTaskId}/complete
Content-Type: application/json

{
  "workerId": "sub-worker-01",
  "variables": {
    "subLmMutationId": { "value": "submut_01HZ", "type": "String" },
    "subLmStatusAfter": { "value": "ACTIVE", "type": "String" }
  }
}

```

If the downstream call fails but is retryable, use `/failure` with retry count and retry timeout. If it is a business rejection, complete into a modeled BPMN branch or throw a BPMN error according to the satellite flow.

14.4 Correlate A Message Catch

Payment, provisioning, work-order, and notification callbacks correlate by operation or external reference:

```

POST /engine-rest/message
Content-Type: application/json

{
  "messageName": "payment-confirmed",
  "businessKey": "subop_01HZ_ACTIVATE_001",
  "processVariables": {
    "paymentId": { "value": "pmt_01HZ", "type": "String" },
    "confirmedAmount": { "value": 5000.00, "type": "Double" }
  }
}

```

Do not use a customer-level correlation key. Use `operationId`, `businessKey`, or the satellite-defined external reference so events for two accounts under the same customer cannot wake the wrong process.

15. Cross-DD Dependencies

DD	Relationship
SUB-LM-01	Subscription master state and state mutation primitives.
DD_SUB-WF-FRAMEWORK-01-WORKERS	Worker catalog, Camunda topics, user tasks, and worker I/O.
FOUNDATION_AUTH	Actor identity, roles, service accounts, and user-task candidate groups.
FOUNDATION_CAMUNDA	BPMN runtime.
FOUNDATION_KAFKA	Kafka event transport.
FOUNDATION_DB	PostgreSQL persistence.
FOUNDATION_DR0OLS	Operator-specific rule packages.
BIL DDs	Billing intents, charge confirmation, and reversals.
FUL DDs	Provisioning, suspension, restriction, disconnection, and HomePass release.
NOT-01	Notification dispatch and retries.

16. Build Order

Implement in this order:

1. Create database migrations for framework tables and indexes.
2. Seed `subscription_operation_config` for initial operators and operation kinds.
3. Implement operation repository with idempotency and in-flight checks.
4. Implement framework start contract used by satellite trigger endpoints.
5. Implement status, history, in-flight, and cancel APIs.
6. Implement operation lifecycle event builders.
7. Implement outbox dispatcher.
8. Integrate Camunda process start and process instance correlation.
9. Add stuck-operation repair job.
10. Add monitoring dashboards and alerts.

17. Test Checklist

- [] Creating an operation inserts one `subscription_operation` row.
- [] Retrying with same idempotency key and same payload returns the same operation.
- [] Retrying with same idempotency key and different payload returns 409.
- [] Two concurrent non-RESTRICT operations on the same subscription cannot both start.
- [] RESTRICT concurrency follows the satellite DD rule.
- [] TERMINATE can interrupt an in-flight non-TERMINATE operation.
- [] Missing config prevents operation start.
- [] Disabled config prevents operation start.
- [] Camunda start success stores `bpmn_process_instance_id`.
- [] Camunda start failure marks operation failed.
- [] Completion sets final state and writes outbox event in the same transaction.
- [] Failure sets failure fields and writes outbox event in the same transaction.
- [] Cancel rejects terminal operations.
- [] Outbox dispatcher publishes pending events.
- [] Outbox dispatcher retries failures and marks failed after retry limit.
- [] Status and history APIs enforce operator scope.

18. Developer Checklist

Before marking this DD implemented:

- [] Tables and indexes exist in migration scripts.
- [] Idempotency request hashing is deterministic.
- [] Operation start uses a database transaction.
- [] Subscription lock is held only during reservation.
- [] External calls are outside locked transactions.
- [] Framework APIs use `FOUNDATION_AUTH`.
- [] Kafka publishing uses `kafka_outbox`.
- [] No worker branches on `operator_code` for business behavior.

- [] Camunda correlation is stored.
- [] Stuck-operation repair exists.
- [] Metrics and alerts exist for outbox and stuck operations.